

跨规模攻防泛化：四方法迁移复现实施方案（v2）

版本 v2，2026-09-09。定位：**可据以直接进入编码与训练的实施方案**，取代同目录 `crossscale_report.md` / `crossscale_report.html`（下称 v1）。

v1 的文献选择与"先迁移复现、再谈改进"的总路线保留。本版做三件事：改正 v1 中与当前代码库不符的事实；把任务从 survival 改写为 **damage 反馈**并重建指标；用**实测数据**替换 v1 中靠推断给出的难度与算力假设。

标注"实测"的数字来自 2026-09-09 在本机（i7-14700KF / RTX 5070 Ti 16 GB / Python 3.10 + torch 2.13）对现有 `had-env` 的直接测量；其余为原论文结果或明确标注的估算。**本文尚未执行任何训练**。

关于本文中的建议值。 文中所有具体数字（训练步数、种子数、评估局数、规模集合等）都是**建议默认值**，不是硬性要求，标有"建议"或列在 §11.4 待确认表中。需要你拍板的都集中在 §11.4。

摘要：v2 相对 v1 的关键修正

#	v1 的表述	实际情况	影响
1	三维、每个红方 27 个离散加速度动作	2D (<code>spatial_dim=2</code> ，工厂默认) 只有 9 个平面动作	DCG 成对收益表从 27×27 降到 9×9，可直接用官方满秩默认；全文"27"需改写
2	任一目标被摧毁即失败，主指标为整局胜率	<code>task_mode="damage"</code> 下目标 不掉血、不摧毁 ，逐步反馈 $-\Delta \text{damage}$ ，整局回报 $-D$	胜率指标作废，须重建为伤害类指标 (§1.4)
3	20v20→50v50 为主实验，隐含"更难"	实测：固定地图下大规模使任务单调变易 。nv1 规则泄漏率 $4v4=0.348 \rightarrow 20v20=0.086$	不能用测试规模的绝对表现说明泛化；必须用同规模锚点归一化 (§1.4、§2.2)
4	现有 REFIL 是可直接复用的复现资产	MARL 源码已在 <code>9e09048</code> 从工作树删除，需从 <code>f8009fe</code> 恢复；且是 自研实现 而非官方移植	影响"复现"表述与工作量估计 (§3.3)
5	主要成本风险是 50v50 的决策延迟	9 动作下 DCG 消息传递量很小；历史管线的真实瓶颈是 数据管线吞吐 ，不是 GPU	实时性降为报告项；管线吞吐值得先量一次 (§2.4)
6	DCG 采用 rank-1 低秩收益	官方 <code>dcg.yaml</code> 的 <code>cg_payoff_rank</code> 默认为空即满秩	9 动作下满秩很便宜，直接用官方默认
7	SPECTra 以 <code>SPECTra_SMACv2</code> 为迁移入口	SMACv2 树里 SPECTra 实现命名为 <code>ss_*</code> ；带 <code>spectra_qmix_plus.yaml</code> 的是 SPECTra_GRF 树，且 GRF 是 固定动作空间 ，与本任务更接近	迁移参照改为 GRF 树 (§4.1)

另有三条 v1 未提及、但对建模有决定性影响的环境事实：**同队碰撞同样致死**（20 m 内双方同归于尽，不分敌我）；**蓝方攻击机开火即自毁**（每架一生只投放一次伤害）；**红方不能悬停** ($v_{\min}=35\text{ m/s}$)。

一、问题建模

1.1 任务的真实反馈

红方控制全部防御单位，蓝方为规则策略。采用 `task_mode="damage"`：

- 目标点**持续存在**，血量每步重置回初始值，只累加受到的伤害，不设上限；
- 每个物理步红方团队奖励 $r_t = -\Delta D_t$ ，蓝方 $r_t = +\Delta D_t$ ；
- 终局不另加奖励**，整局未折扣回报恰为 $-D$ ， $D = \sum_t \Delta D_t$ 为累计伤害；
- 自然终止只有一个条件：**蓝方攻击机全灭**。红方全灭不终止；
- `max_steps` 是采样截断，`bootstrap_mask=1`；自然终止为 0。

这个设计有个很好的性质：**目标函数就是 return，不需要代理指标**；同时它逐步稠密（伤害事件散布全局），不是终局稀疏奖励。这直接影响“是否需要奖励塑形”的判断（§11.1）。

1.2 形式化

Dec-POMDP，红方受控，蓝方并入转移：

$$\mathcal{M}(c) = \langle \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^{N_R}, P, \{\pi_B\}, r, \{\Omega_i\}, \gamma, \langle \mathcal{A}_i = \{0, \dots, 8\} \rangle \rangle$$

$c = (K, N_R, N_B)$ 为规模三元组。团队奖励对全体红方相同——**不能按队员求和**，环境返回的 `team_reward` 已是团队量。

不存在开火动作：红方对 200 m 内蓝方攻击机自动开火，蓝方对 200 m 内目标自动开火。策略学的纯粹是**机动**。

1.3 规模外推的定义

本研究的目标（按你的设定）：**在有限训练资源下训练，泛化到训练中未出现过的对抗规模**，规模同时包含红蓝人数与目标点数。

因此约束只有一条：

测试配置必须没有在训练中出现过。

训练用什么规模、是否多规模混训、是否用课程，都是可自由设计的变量，而且“训练分布怎么选”本身就是值得研究的因子（§10.2 的 F2）。训练完成后冻结参数直接评估，不在测试配置上继续学习、不按测试结果选检查点。

网络还须满足两条工程性质，用断言检查而非实验验证：实体顺序不变性（打乱其他实体存储顺序，同一红方的 9 个动作值不变）与数量接口兼容（换规模不新增、不重训权重）。这两条通过**不等于**规模外推成功。

1.4 指标体系

基础量：整局累计伤害 $D(c)$ ，越小越好。

一级归一化——泄漏率。每架蓝方攻击机开火后自毁，单机伤害贡献有界（同时命中多个目标时可略超 1）：

$$\rho(c) = \frac{\mathbb{E}[D(c)]}{N_B}$$

ρ 近似为“平均每架来袭蓝机成功投放的伤害份额”， $1 - \rho$ 近似拦截率。它消除了 N_B 变化的平凡缩放。

二级归一化——锚点相对得分。 ρ 仍非尺度不变：实测显示任务难度本身随规模强烈变化（§2.2）。故对每个测试配置 c 都测两条**同规模锚点**： $\rho_{\text{rand}}(c)$ （随机加速度）与 $\rho_{\text{rule}}(c)$ （nv1 整数规划规则）。定义

$$\text{NDS}(c) = \frac{\rho_{\text{rand}}(c) - \rho(c)}{\rho_{\text{rand}}(c) - \rho_{\text{rule}}(c)}$$

0 = 与随机同水平， 1 = 追平 nv1 规则， >1 = 超过规则。**建议主表以 NDS 为准，同时给出 D 、 ρ 与两条锚点原值**，让读者能自行还原。

不建议用“相对训练规模的保持率” $\rho(c)/\rho(c_0)$ 作主指标：当 c 比 c_0 容易时它会给出虚假的正向结论。

辅助诊断量（逐局记录，用于失效归因）：红方存活数与同队碰撞次数（高密度拥挤失效）、按目标分解的伤害与首次伤害时刻（是否长期遗漏某目标）、蓝方被击落/自毁/相撞的分解（拦截是主动完成还是蓝方自耗）、动作分布熵与零动作占比（策略是否退化）、回合长度。

二、环境与目标

2.1 环境事实基线（实测核对）

底层是独立仓库 `E:/Code/Open_Score_HAD_Workbench` 的 `had-env`；`Open-SCORE` 只做再导出与规则策略封装，没有第二份物理代码。

项目	事实	出处
空间	2D 为工厂默认；内部保留 3 个坐标槽， z 固定 100 m，垂直分量恒为 0	factory.py
动作	9 个平面离散加速度 （零动作 + $\{-1,0,1\}^2$ 归一化）；索引 0 为 no-op	actions.py:25
动作合法性	存活时 9 个全合法；死亡时只有 0 合法且被强制置 0	environment.py:95
观测	实体行矩阵 $(N_R+N_B+K-1) \times 11$ ：相对位置 3 + 相对速度 3 + 血量 + 存活 + 红/蓝/目标标志； 不含自身行	config.py:132、env/env.py:397
掩码	entity_mask / agent_mask / action_mask / bootstrap_mask 均在 info	environment.py:215
地图	$x,y \in [-2500,2500]$ m， 固定 ，不随人数变化	config.py:103
时间	1 步 = 1 s；max_steps 默认 100；实测自然回合长度 35–61 步	config.py:127
速度	$v \in [35,120]$ m/s， 有最小速度、不能悬停 ； $a_{\max}=40$ ；蓝方系数当前为 1.0	config.py:54
开火	全自动，200 m 内触发，200–400 m 线性衰减	world.py:279、config.py:61
伤害	满伤 1.0/次； 攻击机开火后自毁	agents/attack.py:21
碰撞	扫掠最近距离 ≤ 20 m， 双方同时死亡，不分敌我	world.py:236
出生	目标在 $x \in [-2300,-1900]$ 、 $y \in [-1200,1200]$ 随机；红方在随机选中目标周围 800–2200 m 环带；蓝方 $x \in [0,2500]$ 、 y 全场	env/env.py:321
蓝方	规则分层：上层 reactive/concentrated/balanced（每 5 步或伤亡时重分配），下层 rush/split_rush	grouping/opponents.py

三条对建模影响最大的事实：

同队碰撞致死。 world.py 的碰撞检测遍历所有存活配对，不检查阵营。红方越密集自相撞损失越大，"聚集防守"存在内生成本。

蓝方一次性投放。 每架蓝机最多贡献一次伤害，这给了 $\rho = D/N_B$ 明确物理含义，也解释了回合总在 35–61 步自然结束。

不能悬停。 "守在目标点上方"不是可行策略，只能绕飞，这对策略形态有实质影响。

2.2 实测难度锚点

条件：task_mode="damage"、spatial_dim=2、max_steps=100、蓝方 reactive + rush、每点 60 局独立种子（seed 9000–9059）。

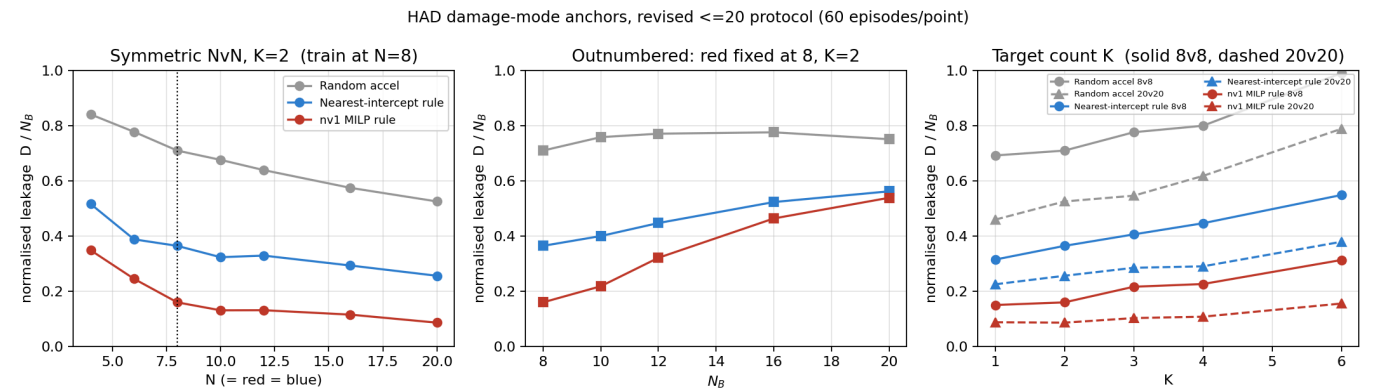


图 1：三条参考策略在三条轴上的归一化泄漏率 $\rho=D/N_B$ 。左为对称放大（虚线为建议的训练规模上界），中为红方固定 8 时蓝方增援，右为目标数。原始数值见 [figures/scale_anchors_small.json](#)。

对称 N_vN ， $K=2$ ：

N_B	4	6	8	10	12	16	20
随机加速度	0.840	0.777	0.709	0.676	0.639	0.574	0.525
最近威胁直追	0.516	0.388	0.364	0.323	0.329	0.293	0.256
nv1 整数规划	0.348	0.245	0.160	0.131	0.131	0.115	0.086

红方固定 8、蓝方增援， $K=2$ ：

N_B	8	10	12	16	20
随机加速度	0.709	0.758	0.770	0.776	0.750
最近威胁直追	0.364	0.400	0.447	0.523	0.562
nv1 整数规划	0.160	0.218	0.321	0.463	0.538

目标数 K ：

K	1	2	3	4	6
nv1 @ 8v8	0.150	0.160	0.216	0.226	0.313
nv1 @ 20v20	0.087	0.086	0.103	0.108	0.155
随机 @ 8v8	0.692	0.709	0.776	0.799	0.990

三条结论：

- 一、固定地图下同比放大规模使任务单调变易。三条策略全部单调下降；nv1 规则 $4v4=0.348 \rightarrow 20v20=0.086$ 。地图面积固定、蓝方在整个 y 范围出生， N_R 增大直接提高对固定区域的覆盖密度，而单机拦截半径不变。（更大范围的趋势见 [figures/scale_anchors.png](#)，50v50 时 nv1 降到 0.025。）
- 这不是说“训小测大”没有意义——那正是实际需求的方向。它的含义是：在放大方向上，测试配置的绝对表现好不能作为泛化成功的证据，必须与同规模锚点比较。这就是 §1.4 引入 NDS 的原因。反过来它也提供了一个很强的证伪工具：既然任务已知变易，若某方法在放大方向上 NDS 反而下降，那就是表示/接口失效的无歧义证据。
- 二、兵力劣势是真正变难的方向。nv1 在 8v20 的泄漏率 0.538 是 8v8 的 3.4 倍，且红方全部阵亡（`red_left`→0.0）。只改 `blue_count`，零环境改动。
- 三、目标数是一条稳定有效的独立轴。 K 从 1 到 6，nv1 在 8v8 上从 0.150 升到 0.313、在 20v20 上从 0.087 升到 0.155，两个规模下都约 1.8–2.1 倍。它与人数轴解耦，且是你明确要求纳入的维度。

2.3 规模协议（按“训练 ≤ 10 、测试 ≤ 20 ”）

集合	配置	说明
训练池 $\mathcal{C}_{\text{train}}$	$N \in \{4,6,8,10\}$ （对称）， $K \in \{1,2,3\}$	具体怎么从池里采样是设计变量，见 §10.2 的 F2
测试·人数外推	$N \in \{12,16,20\}$ ， $K=2$	未见人数
测试·兵力劣势	$8 \text{v} 12$ 、 $8 \text{v} 16$ 、 $8 \text{v} 20$ ， $K=2$	未见配比，主难度轴

集合	配置	说明
测试 · 目标外推	$K \in \{4,6\}$ ，在 8×8 与 20×20 各取	未见目标数； $20 \times 20 \times K (=6)$ 是人数与目标数同时外推
训练内参照	训练池中的配置	判断“是否训得动”，不计入外推结论

建议训练池保持**对称**（ $N_R=N_B$ ），这样兵力劣势轴就是干净的未见维度。若你希望配比也进训练，可在池里加入若干非对称配置，但那条测试轴的解释力会相应减弱——这是个取舍，列在 §11.4。

关于等密度协议。 更干净的做法是让地图面积随 N 缩放以保持密度恒定，把“规模”与“难度”彻底解耦。但 **AeroPoint** 是 `config.py` 的模块级全局量，被观测归一化（`world_diagonal()`）、出生采样、边界裁剪等十余处直接引用，改成按实例配置需要在外部仓库做贯穿式重构。建议本轮不做，用锚点归一化达到同等解释力；若后续要做，改动范围已定位在 `core/env/env.py`（`self.area` 已存在，只需替换直接引用 **AeroPoint** 的十余处）与 `core/function/Function.py` 的两个归一化函数。

2.4 吞吐与算力的实测事实

纯物理吞吐（单进程，随机动作）：

规模	5v5	10v10	20v20	30v30	50v50
步/秒	1000	452	174	90	40

碰撞检测是 $O(N^2)$ ，吞吐约按 $N^{-1.6}$ 衰减。**训练池全在 $N \leq 10$ ，物理吞吐 450–1000 步/秒，是很宽裕的区间；20v20 只用于评估推理。**

一个值得知道的历史参照。 S1 的 REFIL 在 **4v3** 跑 1,000,123 步用了 **15.06 小时**，即有效 **18.4 步/秒**；而同规模纯物理吞吐约 1000 步/秒。历史管线只跑到物理上限的约 2%，瓶颈在单环境串行推进、逐智能体前向调用和小批量更新，不在 GPU 算力。

按网络规模（实体注意力 $d=128$ 、4 头）粗算，训练相关计算量对 RTX 5070 Ti 不构成压力；决定墙钟的是每步的 Python 与 kernel launch 开销。**含义是：如果照搬历史管线的写法，训练会慢两个数量级；如果用官方 PyMARL 的 `parallel_runner`（`batch_size_run=8`，四个官方配置都默认用它），吞吐会好得多。**

我不给具体的吞吐阈值——那个数该由你在看到第一次实测后定。**建议做的事只有一件：第一个方法跑通后，先测一次端到端步/秒再决定训练预算**，避免按错误的吞吐假设排后面所有实验。

三、总体方案讨论

3.1 四个复现对象与各自要检验的假设

对象	机制类别	在本任务中的假设	主要风险
SPECTra-QMIX ⁺	端到端集合策略	SAQA + 集合超网络 mixer 在实体数变化时保持价值分解质量	原始优势部分来自 SMAC 的 实体定向攻击动作 ，本任务动作固定 9 方向，这部分没有对应物
REFIL-QMIX	端到端集合策略	随机实体分解迫使网络在实体子集上也能预测价值，形成组合泛化	单调 mixer 表达限制；分组语义在密集场景下的有效性
InforMARL	图观测聚合	局部关系算子比全场拼接更易迁移到不同人数	on-policy 样本效率低；全观测下图变密
DCG	结构化动作优化	成对协同项能表达“谁去拦谁”的分工	完整图边数 $O(N^2)$ ；成对分解无法表达三体以上收益； 官方图结构是静态的 (§7.4)

对 SPECTra 的科学风险提示。 它在 SMAC 上的增益之一来自对实体定向动作的置换等变输出头，本任务无对应物，只剩 SAQA 编码器与集合超网络 mixer。这不构成放弃的理由（这两部分本身值得检验），但论文中不能把它的 SMAC 增益当作本任务预期。

对 DCG 的先验应当上调。2D 只有 9 个动作，成对收益表仅 9×9，可直接用官方满秩默认；同时任务高度依赖“分工不撞车”（同队碰撞致死），成对协同项的语义与任务结构非常契合。

3.2 共用协议

- 环境：task_mode="damage"、spatial_dim=2、target_initialization="random"、max_steps=100；
- 蓝方：训练与主测试用 reactive + rush；对手泛化单列（§10.2 的 E3）；
- 奖励：环境原生团队奖励 $-\Delta D_t$ ，建议不加塑形项（理由见 §11.1）；
- 训练预算：以环境物理步数为统一单位；具体数值等第一次吞吐实测后定；
- 种子：建议 3 个独立训练种子；
- 模型选择：在训练池内的独立验证集上按平均 D 选检查点，测试配置结果不参与选型；
- 评估：每个测试配置建议 ≥ 300 局，所有方法与锚点共用同一批开局种子（配对比较）。

3.3 复现策略：每方法独立目录，各自沿用官方实现

现状。MARL 源码已在 9e09048 从工作树删除，可从 f8009fe 恢复完整 S1 栈。历史 REFIL 实现是自研 PyTorch 版，其注释明确写明"inspired by REFIL / SPECTra, not a reproduction of either codebase"——不过对照官方 refil.yaml 可以确认它忠实沿用了官方超参（attn 128 / 4 头 / GRU 64 / mixing 32 / hypernet 128 / $\lambda=0.5$ / buffer 5000 / target 200）。

采用的路线（按你的意见）：每个方法一个独立目录，内部尽量贴近官方实现，只统一对外的训练与评估接口。

这条路线有一个此前没看到的有利事实：REFIL、DCG、SPECTra 三个仓库都是 PyMARL 分支，components/episode_buffer.py、controllers/basic_controller.py、runners/parallel_runner.py、config/default.yaml 的骨架几乎一致。它们对环境的要求只有两种约定：

约定	使用者	环境需提供
实体方案（REFIL 特有）	REFIL；DCG-Entity 与 SPECTra 迁移后也走这套	get_entities()、get_masks()、get_entity_size()、get_avail_actions()、get_total_actions()、step()→(标量团队奖励, done, info)、get_env_info()→{entity_shape, n_actions, n_agents, n_entities, gt_mask_avail, episode_limit}
图方案（InforMARL）	InforMARL	node_obs、adj、agent_id、entity_type 等张量，配 graph_buffer

所以共享层只需要两个环境适配文件，不需要统一算法实现。这正好符合“算法实现相对独立、训练接口统一”的要求。

REFIL 的实体接口契约（读自官方 src/envs/group_matching/group_matching.py 与 src/controllers/entity_controller.py）有几条硬约定，HAD 适配层必须遵守：

- 受控智能体必须排在实体列表最前 n_{agents} 位（entity_controller 用 [:n_agents] 切片写入 actions_onehot）；
- obs_mask[i,j]=1 表示 i 看不到 j ；entity_mask[i]=1 表示实体 i 不存在；
- n_entities、n_agents 是整个运行的固定上界（buffer 按上界分配），变长靠掩码实现；
- step() 返回标量团队奖励——与 damage 模式天然吻合；
- 我们没有真值分组，故 gt_mask_avail=False，用官方 refil 而非 refil_group_matching。

必须在论文中写明：这些是在官方实现基础上的场景迁移，不是官方结果的原样复现。缓解措施：每个方法先在其原始基准的小任务上做原生 sanity check，再迁移；逐项记录与原实现的差异清单；超参以各自原配置为起点。

3.4 优先级与降级

建议优先级 REFIL > DCG > InforMARL > SPECTra：REFIL 有历史资产且官方超参已核对，风险最低，作为必成基线；DCG 在本任务先验期望最高且实现最独立；InforMARL 代表另一大类（图 + on-policy）；SPECTra 因动作空间不匹配，边际信息量最低，列为第一顺位可裁项。

若进度紧张，建议砍方法而不是砍种子或砍评估——三方法 × 3 种子的完整结果好于四方法 × 1 种子。

四、方法一迁移：SPECTra-QMIX⁺

官方定位：[SPECTra, 2025](#)，[代码](#)。

4.1 迁移参照选 GRF 树，不是 SMACv2 树

仓库有两棵独立的树。核对文件清单后：

- SPECTra_SMACv2/ 中 SPECTra 实现命名为 `ss_*` (`modules/agents/ss_rnn_agent.py`、`modules/layer/ss_attention.py`、`modules/mixers/ss_mixer.py`、`controllers/ss_controller.py`、`config/algs/ss_qmix.yaml`)，没有 `spectra_qmix_plus.yaml`；
- SPECTra_GRF/ 才有 `modules/agents/spectra_rnn_agent.py`、`modules/layer/spectra_attention.py`、`modules/mixers/spectra_mixer.py`、`controllers/spectra_controller.py`、`config/algs/spectra_qmix_plus.yaml`。

而且 GRF 是固定动作空间的环境（足球动作集与实体编号无关），与本任务的 9 个固定加速度方向结构相同；SMACv2 则带实体定向攻击动作。因此迁移参照应取 SPECTra_GRF。v1 指向的 SPECTra_SMACv2/scripts/.../spectra_qmix_plus.sh 入口需要更正。

官方 `spectra_qmix_plus.yaml` 的关键项：`mac=spectra_mac`、`agent=spectra_rnn`、`mixer=spectra_mixer`、`learner=nq_learner`、`hidden_size=64`、`n_head=4`、`mixing_n_head=4`、`hypernet_embed=32`、`lr=0.001`、`td_lambda=0.6`、`optimizer=adam`、`use_SAQA=True`、`batch_size_run=8`、`buffer_size=5000`、`batch_size=32`、`epsilon_anneal_time=500000`。注意 `obs_agent_id: False`、`obs_last_action: False`——不含固定长度的智能体 ID，这对规模外推是有利的默认。

4.2 场景差异与迁移动作

维度	GRF 原场景	HAD 本场景	迁移动作
实体类型	我方球员、对方球员、球	红方、蓝方、目标点	三类类型嵌入；目标点必须独立成类，不能并入敌方
动作	固定 19 类足球动作	固定 9 类加速度	换输出维度，结构不变
实体数	固定	跨 episode 变化	按上界分配 + <code>entity_mask</code>
奖励	进球 + 塑形	ΔD_t 标量团队奖励	直接替换
回合	固定长度	蓝方全灭自然终止或截断	正确区分终止与截断
课程	<code>SPECTra_SMACv2/run/curriculum.py</code>	由我们的训练池定义	不用官方课程模块，用统一训练池

保留：SAQA、循环记忆、ST-HyperNet 集合超网络 mixer、非负权重单调性。移除：无对应语义的实体定向动作分支。

4.3 迁移后的关键验证

1. **SAQA 置换不变性**：固定某红方，随机打乱其余实体行顺序（同时打乱掩码），其 9 个 Q 值应逐元素一致（容差 $1e-5$ ）。
2. **Mixer 顺序对齐**：构造两个红方个体 Q 明显不同的输入，交换它们在个体 Q 向量与超网络实体输入中的位置， Q_{tot} 应不变；若只交换其一则应改变——这条能抓住“个体 Q 与超网络红方顺序错位”这个最容易犯的 bug。
3. **单调性**：对随机输入数值检查 $\partial Q_{tot} / \partial q_i \geq 0$ 。
4. **变数量前向**：同一份权重在 $N=4$ 与 $N=20$ 、 $K=1$ 与 $K=6$ 下都能前向且不新增参数。
5. **原生 sanity check**：在 GRF 的 `academy_3_vs_1_with_keeper` 上短跑，确认迁移前的实现链路可用。

五、方法二迁移：REFIL-QMIX

官方定位：[REFIL, ICML 2021](#)，[代码](#)。同时是基线与已有资产的重新验证。

5.1 官方配置（已核对 `src/config/algs/refil.yaml`）

```
agent=imagine_entity_attend_rnn、mac=entity_mac、mixer=flex_qmix、learner=q_learner、runner=parallel、
batch_size_run=8、training_iters=8、buffer_size=5000、target_update_interval=200、rnn_hidden_dim=64、
attn_embed_dim=128、attn_n_heads=4、mixing_embed_dim=32、hypernet_embed=128、
softmax_mixing_weights=True、double_q=True、lmbda=0.5、entity_last_action=True、
epsilon_anneal_time=500000。
```

历史 S1 的超参与此**逐项一致**，说明历史实现忠实沿用了官方配置，可作为交叉验证的参考实现。

关键源文件：`src/modules/agents/entity_rnn_agent.py`、`src/modules/layers/attention.py`、`src/modules/mixers/flex_qmix.py`、`src/controllers/entity_controller.py`、`src/learners/q_learner.py`。

5.2 训练目标

$$Q_{\mathrm{tot}} = f_{\mathrm{mix}}(q_1, \dots, q_{N_R}; s), \quad \frac{\partial Q_{\mathrm{tot}}}{\partial q_i} \geq 0; \quad \mathcal{L} = (1 - \lambda) \mathcal{L}_{\mathrm{TD}} + \lambda \mathcal{L}_{\mathrm{aux}}, \quad \lambda = 0.5.$$

辅助分支按随机实体分组构造 imagined value，拟合同一个**真实 TD 目标**。分组在 episode 内保持一致，各分支独立维护循环历史，共享网络参数。部署只用完整观测分支。不能把辅助分支改成自行设计的局部奖励。

5.3 场景差异与迁移动作

维度	SMAC / group_matching	HAD	迁移动作
实体特征	单位类型、血量、护盾、相对位置	相对位置/速度、血量、存活、三类标志、(可选) 自身绝对量	在 <code>features.py</code> 定义 schema，四方法共用
动作	移动 + 攻击第 j 个敌人	固定 9 类	输出维度 9；无实体定向分支
<code>gt_mask</code>	<code>group_matching</code> 有真值分组	无	<code>gt_mask_avail=False</code> ，用 <code>refil</code> 不用 <code>refil_group_matching</code>
观测范围	视野受限	全场可观测	<code>obs_mask</code> 全 0（对存在实体）
奖励	SMAC 密集塑形	$-\Delta D_t$	直接替换

5.4 迁移步骤与验证

- 1. 写 `HADEntityEnv` 实现 §3.3 的实体契约，注册进 REFIL 的 `src/envs/__init__.py`；
- 2. 用官方 `group_matching` 环境短跑，确认恢复/vendored 的 REFIL 链路可用；
- 3. **验证 A**：随机分组的形状与时序一致性——同一 episode 内 `group_a` 不变；within/interaction 两条可见性掩码互补且都不含已死实体；
- 4. **验证 B**：置换不变性与 mixer 顺序对齐（同 §4.3 的 1、2）；
- 5. **验证 C**：`entity_last_action=True` 时，非智能体实体的动作 one-hot 恒为 0；智能体的是**上一步**动作而非当前步（官方 `entity_controller._build_inputs` 的切片语义，易错）；
- 6. 按共用协议在训练池上训练，冻结后评估。

旧小规模 **checkpoint 不得热启动并入主结果**；若要测试，作为独立对照单列。

六、方法三迁移：InforMARL

官方定位：InforMARL, ICML 2023, [代码](#)。UniMP 图聚合 + RMAPPO；图用于**观测表示**，动作由 actor 直接输出，不接 mixer。

关键源文件：`onpolicy/algorithms/graph_actor_critic.py`、`onpolicy/algorithms/utils/gnn.py`、`onpolicy/algorithms/graph_mappo.py`、`onpolicy/runner/shared/graph_mpe_runner.py`、`onpolicy/utils/graph_buffer.py`、场景模板 `multiagent/custom_scenarios/navigation_graph.py`、入口 `onpolicy/scripts/train_mpe.py`。

6.1 场景差异与迁移动作

维度	navigation_graph	HAD	迁移动作
任务	每个 agent 有指定 goal，导航避碰	集体防御多个共享目标	删除预分配 goal 条件，所有目标进入实体图。不能给 InforMARL 额外提供"正确分配"答案，否则对其他方法不公平
动作	二维离散/连续加速度	9 类平面加速度	换输出头，沿用动作映射
节点类型	agent / landmark / obstacle	红方 / 蓝方 / 目标点	三类 entity_type
建边	感知半径	全场可观测	见下
agent 数	固定	跨 episode 变化	graph_buffer 按上界分配 + active mask
奖励	每 agent 个体奖励	标量团队奖励	全体共享同一团队奖励；critic 用团队价值

建边策略建议跑两个配置：

配置	规则	用途
FullObs（主对照）	全部存在实体两两可见	信息条件与其他三方法一致，可公平比较
kNN-\$k\$（第二配置，建议 \$k=10\$）	每节点连最近 \$k\$ 个实体	度数固定 $\Rightarrow$ 表示与计算量对规模天然不变

这个对照几乎不增加成本，却是本方法在本课题下最有价值的信息。**不能用 kNN 的速度优势去和别人的全观测结果比性能。**

6.2 样本预算的公平性

InforMARL 是 on-policy，样本效率显著低于三个 off-policy 方法。建议：主表按相同环境步数比较，同时单列"各方法学习曲线 vs 环境步数"图；若在共同预算下明显未收敛，追加一个更长的 InforMARL 单点，标注为不等预算参照，不并入主表。

6.3 迁移后的关键验证

- 1. **图批处理正确性**：官方自带 `onpolicy/algorithms/utils/test_gnn_batching.py`，先用它确认 vendored 代码的批处理没被改坏。
- 2. **置换不变性**：打乱节点顺序（同步打乱邻接矩阵），同一 agent 的动作分布不变。
- 3. **死亡处理**：死亡单位不进入 actor loss；图池化分母不含 padding；全亡时终止边界正确。
- 4. **变数量**：同一权重在 $N=4$ 与 $N=20$ 下都能前向；`graph_buffer` 的 active mask 与实际人数一致。

七、方法四迁移：DCG

官方定位：[DCG, ICML 2020](#)，[代码](#)。**结构化价值模型 + 在线组合优化**，执行时不调用任何环境副本。

关键源文件：`src/controllers/dcg_controller.py`、`src/learners/dcg_learner.py`、`src/modules/agents/rnn_feature_agent.py`、`src/config/algs/dcg.yaml`；原生 sanity check 环境 `src/envs/stag_hunt.py` 配 `src/config/envs/rel_overgen.yaml`。

7.1 官方配置（已核对 `dcg.yaml`）

```
cg_edges='full'、msg_iterations=8、msg_anytime=True、msg_normalized=True、cg_payoff_rank: 空（满秩）、
cg_utilities_hidden_dim: 空、cg_payoffs_hidden_dim: 空、duelling=False、mixer: 空、agent='rnn_feat'、
mac='dcg_mac'、learner='dcg_learner'、buffer_size=500、target_update_interval=200、
epsilon_anneal_time=50000。
```

注意 buffer 与 ϵ 退火与 REFL 差一个量级，各方法保留各自官方配置，不要互相拉平。

7.2 价值分解

图节点是红方**动作变量**；蓝方与目标点只进观测，不作为求解节点。

$$u_i(a_i)=f^V_{\theta(a_i\mid h_{i,t})},\quad p_{ij}(a_i,a_j)=\frac{1}{2}\bigl[f^E_{\phi(a_i,a_j\mid h_{i,h_j})}+f^E_{\phi(a_j,a_i\mid h_{j,h_i})}\bigr],$$
$$Q_{\theta}(\tau_t,\mathbf{a})=\frac{1}{n}\sum_{i\in V}u_i(a_i)+\frac{1}{|E|}\sum_{\{i,j\}\in E}p_{ij}(a_i,a_j).$$

端点交换时动作维度同时交换，实现上第二张表需转置。

消息更新：

$$m_{i\rightarrow j}(a_j)=\max_{a_i\in\mathcal{A}_i}\Bigl[\frac{u_i(a_i)}{n}+\frac{p_{ij}(a_i,a_j)}{|E|}+!\sum_{k\in\mathcal{N}(i)\setminus j}!$$
$$m_{k\rightarrow i}(a_i)\Bigr].$$

完整图有环，固定 8 轮**不提供全局最优保证**；沿用官方逐轮联合动作评价，返回已检查轮次中学习价值最高的动作（原附录 Algorithm 3）。

成本（9 动作下）：20 个红方 190 条边，每步表项运算量约 $2\times 8\times 190\times 81\approx 2.5\times 10^5$ ；即使 50 个红方（1225 条边）也只有 1.6×10^6 。批量张量实现下 GPU 上可忽略，**DCG 的延迟风险在本任务中基本消失**。

7.3 场景差异与迁移动作

维度	SMAC / stag_hunt	HAD	迁移动作
观测输入	固定长度扁平向量	变长实体表	把 <code>rnn_feature_agent</code> 的输入层换成实体注意力编码器（记为 DCG-Entity ），输出仍是隐状态
智能体身份	随人数变化的 one-hot ID	不能用	去掉 ID one-hot，保留固定维物理属性与存活标记
动作	SMAC 动作集	9 类	换维度
奖励	团队奖励	$-\Delta D_t$	直接替换

不加 REFL 的辅助分组损失，不保留 QMIX mixer，不学习新的构图器。实体注意力在这里只解决输入长度问题。

7.4 一个具体的迁移阻塞点：官方图结构是静态的

官方 `dcg_controller` 在 `__init__` 里按 `args.n_agents` **一次性构建边集**。我们的训练池跨 episode 变化人数，因此必须改成**按 episode 重建图索引**，或按上界建图 + 节点/边 padding mask。这是 DCG 迁移中最实质的一处改动，必须在计划里单列。

配套约定：图在每局重置时按初始 n 建立并**固定到底**；死亡节点保留、只有动作 0 合法，不因死亡重构图；归一化分母保持初始 $n_i|E|$ ，padding 不参与求和；消息归一化必须排除 $-\infty$ 位置与 padding，否则产生 NaN；每个存在节点至少保留一个合法动作。

7.5 STAR 对照

按本局初始红方索引固定根节点 r ， $E=\{r_j:j\neq r\}$ 。树上 Max-Sum 的汇集与条件回溯给出对**当前学习价值模型**的精确最大化：

$$M_j(a_r) = \max_{a_j \in \mathcal{A}_j} \left[\frac{u_j(a_j)}{n} + \frac{p_{\{r\}}(a_r, a_j)}{n-1} \right], \quad a_r^* = \arg \max_{a_r} \left[\frac{u_r(a_r)}{n} + \sum_{j \neq r} M_j(a_r) \right], \quad a_j^* = B_j(a_r^*)$$

保证的对象是**学习到的价值**，不是真实任务的最优防御。平局时必须按已选根动作回溯叶动作，不能各节点独立打破平局。STAR 用于分离“求解成本”与“表示容量”，不作主配置。

7.6 迁移后的关键验证

- 求解器正确性**：在很小的人工收益表（例如 3 个节点、3 个动作）上，用穷举核对 STAR 树解码是否给出真正的最大值；完整图则记录返回动作的学习价值与消息预算。这是纯数值检查，不需要环境。
- 对称化正确性**：随机构造 f^E_ϕ ，检查 $p_{\{ij\}}(a,b) = p_{\{ji\}}(b,a)$ 。
- 归一化不变性**：同一物理局面下， n 与 $|E|$ 的归一化使 Q_Θ 的量纲不随人数线性放大——用两个不同 n 的构造输入检查数量级。
- 变量量建图**：连续跑 $N=4,6,8,10$ 的 episode，确认图索引每局正确重建、循环状态不串号。
- 原生 sanity check**：官方 `rel_overn` 小任务上短跑，确认完整图学习链路可用（不需要 SC2）。

八、整体代码架构安排

8.1 布局

挂在现有 `Open-SCORE/src/open_score/` 下，不新建平行仓库。共享的只有环境适配层与评估协议；四个算法各自独立。

```
Open-SCORE/src/open_score/crossscale/
├── had_env/                    # 唯一共享的环境适配层
│   ├── features.py           # 实体 schema、9↔27 动作映射、掩码、归一化（四方法共用）
│   ├── entity_env.py         # HADEntityEnv: REFIL 实体契约（REFIL / DCG / SPECTra 共用）
│   ├── graph_env.py          # HADGraphEnv: InforMARL 的 node_obs/adj/entity_type 契约
│   └── scales.py              # 训练池与测试集的规模分布定义
├── methods/
│   ├── refil/                 # vendored REFIL（裁剪）+ HAD 接入 + 差异清单
│   ├── dcg/                   # vendored dcg（裁剪）+ DCG-Entity 改动 + 动态建图
│   ├── informarl/             # vendored InforMARL onpolicy（裁剪）
│   └── spectra/                # vendored SPECTra_GRP（裁剪）
├── protocol/
│   └── policy_api.py          # 统一策略接口: load(ckpt) -> reset()/act(state, side,
│       action_ids)
│   ├── anchors.py             # 规则锚点，直接复用 coverage_rule.run_episode
│   ├── evaluate.py            # 冻结评估、四条测试轴、配对种子、逐局落盘
│   └── report.py               # 主表与图
└── configs/                   # 每方法一个 yaml，尽量贴近官方原值
```

8.2 统一接口只有两个

训练侧：每个方法目录暴露 `train(cfg) -> checkpoint_path`。内部完全按各自官方框架运行（PyMARL 的 `main.py/run.py` 或 InforMARL 的 `train_mpe.py`），我们只包一层入口。

评估侧：每个方法目录暴露 `load_policy(checkpoint) -> Policy`，`Policy` 提供 `reset()` 与 `act(state, side, action_ids)`，即现有 `register_end_to_end_policy` 的契约。注册后即可复用 `run_episode / evaluate` 与网页回放器，与规则锚点走完全相同的评估路径——这是保证锚点可比的关键，不要另写评估循环。注意该接口要求把模型的 0-8 动作索引通过 `action_ids` 映射到原生 27 编号。

8.3 与现有代码的衔接

- 环境入口**：用 `HADStage3Adapter` 而非 `Parallel API`。Parallel API 要求同时提供蓝方动作，而规则蓝方逻辑在 adapter 的 `commanded_rule_actions` 里；adapter 的 `step(red_actions)` 省略蓝方参数即自动执行规则蓝方，正是训练需要的形态。

- **观测**：建议自建实体特征而非直接用 Parallel 的 11 维行——原生观测不含自身绝对状态（自身速度只能从静止目标行间接反解），也缺少存活计数、剩余时间等上下文。历史 S1 也是自建的（ENTITY_DIM=12、SELF_DIM=10）。特征表在 features.py 集中定义。
- **实体上界**：n_entities 与 n_agents 按训练池与测试集的并集取上界（例如 20 红 + 20 蓝 + 6 目标 = 46 实体、20 智能体），buffer 按上界分配，变长靠掩码。

九、实现细节注意

按踩坑概率排序。前六条是历史项目已出现过或环境文档明确警告的。

1. **动作编号双轨制**。分组适配器保留原生 27 编号，2D 只用其中 9 个平面方向；训练接口用 0–8。两套编号不能混用，在 features.py 集中映射并加断言。
2. **截断 vs 自然终止的 bootstrap**。damage 任务只有“蓝方攻击机全灭”是自然终止（bootstrap_mask=0）；max_steps 到时是截断（bootstrap_mask=1，必须 bootstrap）。实测自然回合 35–61 步，截断很少触发，但一旦触发就是长尾困难局面，恰恰不能算错。
3. **团队奖励不可求和**。环境返回的队友 team_reward 已是团队量。按队员相加会放大 N_R 倍，且放大倍数随规模变化，直接污染跨规模比较。PyMARL 的标量团队奖励约定与此天然吻合。
4. **死亡单位保留槽位并仍接收团队奖励**。damage 任务下固定槽保留到整局结束，死亡槽零观测、强制作 0。agent_mask 用于 actor 损失，bootstrap_mask 用于团队 critic。死亡当步的更新使用动作产生前的存活掩码。
5. **循环状态必须跟随持久实体 ID**。不能随数组下标路由，否则单位死亡后隐状态会串给其他单位。
6. **entity_last_action 的时序**。REFIL 的 entity_controller._build_inputs 写入的是上一步动作，且只写前 n_{agents} 个实体；照搬时不要错位成当前步。
7. **队友碰撞是内生成本**。20 m 内双方同归于尽且不分敌我。“全体扑向同一威胁”会自伤，训练早期可能出现红方大量自相撞。这是任务的真实约束，不要当 bug 修掉，但要在诊断里单独统计——它很可能是高密度下的主要失效模式。
8. **没有悬停**。 $v_{\min}=35$ m/s，红方必须持续移动，只能绕飞。
9. **观测归一化统计要冻结**。环境内建的位置/速度归一化与人数无关，这点很好。若在 features.py 里再加任何基于数据统计的归一化，必须在训练结束时冻结并随 checkpoint 保存，评估时不得重新估计。
10. **蓝方随机性是方差来源**。reactive 每 5 步或伤亡时重采样目标分配，opponent_rng 由 seed ^ 0x375AC18F 派生。评估时所有方法必须共用同一批开局种子做配对比较。
11. **DCG 的动态建图与 $-\infty$** 。见 §7.4。填充边不得进入求和；均值归一化必须排除 $-\infty$ 与 padding。
12. **重排与规模的运行时断言**。写成可开关的检查而非独立测试项目：打乱实体顺序 Q 值不变；重排智能体索引输出按同一排列重排；同一 checkpoint 在训练池最小配置与测试集最大配置下都能跑完整局且不新增权重。

十、实验设计

10.1 实验设置

项	建议值
环境	task_mode="damage"、spatial_dim=2、target_initialization="random"、max_steps=100
训练池	$N\in\{4,6,8,10\}$ 对称、 $K\in\{1,2,3\}$ ；蓝方 reactive + rush
测试集	§2.3 的三条轴，全部未在训练中出现
训练预算	以环境物理步数计；具体值待第一次吞吐实测后定
种子	3 个独立训练种子

项	建议值
模型选择	训练池内独立验证集按平均 $\$D\$$ 选优；测试结果不参与选型
评估	每配置 ≥ 300 局，全方法共用同一批开局种子
主指标	$\mathrm{NDS}(c)$ ，附 $\$D\$$ 、 ρ 及两条锚点

10.2 实验类型

- E0 — 原生 sanity check（每方法必做，不进主表）。** 各自原始基准的最小任务：REFIL 用 `group_matching`，DCG 用 `rel_overnen`，SPECTra 用 GRF `academy_3_vs_1_with_keeper`，InforMARL 用 `navigation_graph` + 自带的 `test_gnn_batching.py`。只判断实现链路对不对。
- E1 — 训练池内性能。** 学习曲线与最终 $\$D\$$ 。**这是后续结论的前提：**若某方法在训练池内就打不过同规模 nv1 规则，它在外推轴上的表现不能被解释为"规模泛化失败"。
- E2 — 三条外推轴（主实验）。** 人数外推、兵力劣势、目标外推，冻结参数直接评估。人数外推方向已知变易，故在该方向上 NDS 下降即表示失效的无歧义证据。
- E3 — 对手泛化（低成本、高价值）。** 训练只用 `reactive + rush`，测试扩展到 `concentrated`、`balanced` 上层与 `split_rush` 下层的组合。**不需要任何新代码**（`run_episode` 已支持），却能显著增强完整性——规模泛化与对手泛化是审稿人会一起问的两个问题。
- E4 — 两个因子的交叉（本方案的科学核心之一）。**
- **F1 = 架构：**四个方法。
 - **F2 = 训练分布：**至少两档——**单一规模**（只 8v8、 $K=2$ ）与**多规模混合**（训练池全采样）。可选第三档：**由小到大的课程**。

F2 直接回答你关心的"有限训练资源下怎么训才最能外推"，而且它对**所有**方法都适用，是一个方法无关的结论，比单纯的方法排名更有普适价值。成本控制：F1×F2 全交叉是 4×2=8 组；若时间紧，先对全部方法跑多规模混合（主设定），再只对表现最好的两个方法补单一规模对照。

- E5 — 消融。** 只对表现最好的一到两个方法做：REFIL 去掉随机分组辅助损失（退化为 `qmix_atten`）；DCG 完整图 vs STAR vs 无成对项；InforMARL FullObs vs kNN-10。
- E6 — 诊断。** §1.4 的辅助量按测试配置分解。重点：同队碰撞是否随密度上升、是否存在长期遗漏的目标、动作分布是否退化。

10.3 预计时间

训练池全在 ≤ 10 ，物理吞吐 450–1000 步/秒，**训练成本很低**。评估在 ≤ 20 ，最慢的 20v20 约 174 步/秒（纯物理）、加规则锚点约 0.28 s/局。

不给具体小时数，因为它完全取决于管线实现质量（历史管线与物理上限差 50 倍）。给出的是**排序**：锚点与评估的成本是确定的（锚点已测大半，评估约 4 方法 × 3 种子 × 约 11 个配置 × 300 局）；训练成本是不确定项，第一个方法跑通后测一次即可锁定后续排期。

10.4 主结果表模板

方法	训练分布	测试配置	$\$D\$$	ρ	ρ_{rand}	ρ_{rule}	NDS	种子区间
REFIL-QMIX	混合 ≤ 10	12v12 K2	待测	待测	0.639	0.131	待测	待测

方法	训练分布	测试配置	D	ρ	ρ_{rand}	ρ_{rule}	NDS	种子区间
REFIL-QMIX	混合 $N \leq 10$	20v20 K2	待测	待测	0.525	0.086	待测	待测
REFIL-QMIX	混合 $N \leq 10$	8v20 K2	待测	待测	0.750	0.538	待测	待测
REFIL-QMIX	混合 $N \leq 10$	8v8 K6	待测	待测	0.990	0.313	待测	待测
REFIL-QMIX	混合 $N \leq 10$	20v20 K6	待测	待测	0.788	0.155	待测	待测
... (DCG / InforMARL / SPECTra 同结构)								

锚点列已实测填入（60 局），正式评估时补到 300 局并更新。

十一、其他讨论与总结

11.1 是否引入奖励塑形、课程学习、模仿学习

奖励塑形：建议主协议不加。 三条理由。第一，damage 奖励本身逐步稠密，伤害事件散布全局，不是“只有终局才有信号”的稀疏问题。第二， $-\Delta D_t$ 就是要优化的目标，塑形会引入“优化的东西和评价的东西不一致”的解释负担；而跨规模比较对这种不一致特别敏感——塑形项的量纲通常随人数变化（如“到最近威胁的距离和”），会在不同规模上引入不同偏置。第三，塑形是审稿人质疑公平性的高频靶点。

若 E1 显示所有方法都远不及 nv1 规则，再考虑**势函数塑形** ($F = \gamma \Phi(s') - \Phi(s)$)，它保证最优策略不变，是唯一理论上“免费”的形式；届时四方法统一启用并做消融。

课程学习：现在是一个正当的研究变量。 既然训练规模可自由设计，“由小到大的课程”就是 E4 中 F2 的一个合法档位，与“单一规模”和“多规模混合”并列比较。需要守住的只有一条：**课程覆盖的规模都算训练过，测试集必须在课程之外**。这也解释了为什么 SPECTra 原文的课程结果不能直接搬用——它的课程包含了目标规模，而我们的测试规模不在训练分布里。

模仿学习：可用，但要放对位置。 环境里有现成的强专家：nv1 整数规划规则（8v8 泄漏率 0.160，20v20 为 0.086），可无限量低成本生成演示（实测约 0.28 s/局）。BC 预热能显著缩短从零学习的时间。

代价是：BC 会把所有方法拉到同一个规则策略的邻域，**可能系统性压缩方法之间的差异**，而方法差异正是要测量的东西；而且规则策略本身有自己的规模特性（放大时变好、劣势时变差），BC 会把这个特性传染给所有学习方法，让“架构的规模泛化能力”难以分离。

建议：**主协议不用 BC**；作为**预先声明的降级方案**——若中期发现多数方法在训练池内都打不过规则，则四方法**统一**启用相同的 BC 预热（同一演示集、同一预热步数），在论文中作为协议变更明确记录并附消融。绝不能只给某一个方法加。

统一的写作原则：任何附加技巧，只要满足“全方法统一启用 + 预先声明 + 有消融”，就不会损伤论文；损伤论文的是只给自己的方法加、或事后为了救某个结果才加。本轮四个对象都是他人的方法，我们没有要偏袒的对象，协议容易保持干净，应当利用这个优势。

11.2 这条路线能支撑一篇二区论文吗

纯复现不够。四个已有方法在新环境上的迁移比较本质是实证研究，二区期刊接受高质量的基准与实证研究，但通常要求超出"跑了四个别人的算法"。

已经具备、可以主张的贡献有三项：

1. **问题与基准。**带真实物理（连续动力学、自动开火、同队碰撞、不可悬停）的多目标攻防跨规模基准，配 damage 反馈、三条正交外推轴与**同规模难度锚点**。锚点这条有方法论价值：图 1 展示的"固定地图下放大规模反而变易"是一个**反直觉且可复现的观察**，说明此前文献中大量"训小测大"实验可能系统性高估了泛化能力。这本身值得单独一节。
2. **系统性实证。**四类机制在同一协议下的公平比较，加上**架构 × 训练分布**的二因子设计（E4）——后者给出的是方法无关的结论，普适性比单纯的方法排名更强。
3. **失效分析。**即使结果是负面的，只要归因清楚、证据充分仍然成立；前提是排除"训练不足"这个平凡解释，这就是 E1 的作用。

还缺一项方法层面的改进，二区审稿人大概率会要。从上面的分析里已自然浮现两个低成本、强动机的候选：

- **固定邻域截断（top-\$k\$ 近邻实体）+ 密度条件化。**让表示与计算量同时对规模不变，直接针对人数外推方向暴露的表示失效。InforMARL 的 kNN 配置已是这个想法的特例，可自然推广到实体注意力与 DCG 的建图。
- **规模感知的价值归一化。**DCG 用 $\$1/n\$$ 、 $\$1/|E|\$$ 归一化，REFIL 的 FlexMixer 不除以人数——不同方法对规模的归一化处理**互不一致且都缺乏依据**，这正是跨规模失效的一个可疑来源。统一且有依据的归一化方案是廉价且有说服力的贡献。

建议定位：基准 + 系统实证 + 由实证直接导出的一个改进。不要定位成"提出一个新的通用 MARL 算法"。

11.3 一个月的进度安排

按你的期望（方法迁移 2–3 天出初步结果，剩下时间做改进与写作）排：

时间	目标	产出
第 1–3 天	had_env/ 共享层：features.py、entity_env.py（REFIL 实体契约）；vendored REFIL 接入；E0 sanity check	REFIL 在训练池上能跑、损失正常；测一次端到端吞吐并据此定训练预算
第 4–6 天	DCG 接入（含 §7.4 的动态建图）+ E0；REFIL 首轮 3 种子训练与评估	REFIL 主表行；DCG 能跑
第 7–10 天	InforMARL（graph_env.py + vendored onpolicy）与 SPECTra 接入 + E0；DCG 3 种子	四方法均可训练；DCG 主表行
第 11–14 天	四方法 3 种子全部完成；E2 三条外推轴 + E3 对手泛化；锚点补到 300 局	主表与主图完整
第 15–18 天	E4 训练分布因子（至少单一规模 vs 多规模混合）；E5 消融；E6 诊断	二因子结果与失效归因
第 19–24 天	改进方案（§11.2）的最小版本与验证	改进的对照结果
第 25–30 天	出图、成稿	论文初稿

判断：可行但没有余量。主要风险不在算力（训练池全在小规模，很便宜），而在三处：MARL 代码需先恢复并接通；四个方法从"能跑"到"训得动"的调试次数不可预测；四周内既完成全部实验又完成写作偏紧。

三条建议：一是第 14 天设一个明确的检查点，主表不完整就按 §3.4 砍方法（SPECTra 第一顺位）；二是改进方案的想法在第 1 周就定下来，这样跑主实验时可以顺带收集它需要的诊断量，不用回头补跑；三是若目标是投稿而非只是跑完，把写作放到第 5–6 周更现实。

11.4 仍需你拍板的问题

#	问题	本文的建议
Q1	训练池是否保持对称（\$N_R=N_B\$）	保持对称，让兵力劣势成为干净的未见轴
Q2	训练分布（F2）是否作为正式实验因子，还是只用多规模混合一档	作为正式因子，至少两档；它是方法无关的结论，性价比高
Q3	训练预算与种子数	预算等第一次吞吐实测后定；种子建议 3 个
Q4	每个测试配置的评估局数	建议 300；若要更窄的区间可提高
Q5	是否投入约一周做等密度（地图随 \$N\$ 缩放）协议	本轮不做，用锚点归一化替代
Q6	四方法是否必须全做	不必须；SPECTra 第一顺位可裁
Q7	是否现在就锁定一个改进方向，以便主实验顺带收集诊断量	建议锁定，倾向 top-\$k\$ 近邻截断 + 规模感知归一化

附：证据与数据来源

- 难度锚点（本方案协议范围）：[figures/scale_anchors_small.json](#) / [.png](#)，60 局/点，seed 9000–9059，2026-09-09 实测。
- 难度锚点（更大范围趋势，含 30v30、50v50、20v50）：[figures/scale_anchors.json](#) / [.png](#)，24 局/点，seed 7000–7023。
- 吞吐：单进程纯物理，随机动作，`max_cycles=100`，同日实测。
- 历史训练成本：S1 REFIL 1,000,123 步 / 15.06 小时，见 [S1/S2 报告](#)。
- 环境事实：[E:/Code/Open_Score_HAD_Workbench/had_env/](#)，协议 `had-workbench-2.1.0`；接口见 [README](#) 与 [Workbench docs/API.md](#)。
- 官方配置核对：REFIL [src/config/algs/refil.yaml](#)、DCG [src/config/algs/dcg.yaml](#)、SPECTra [SPECTra_GRF/config/algs/spectra_qmix_plus.yaml](#)；实体契约读自 REFIL [src/envs/group_matching/group_matching.py](#) 与 [src/controllers/entity_controller.py](#)（2026-09-09 取自各仓库默认分支）。
- 历史 MARL 源码：Git [f8009fe](#)（完整 S1 栈）、[9e09048^](#)（v6）。
- v1 方案原文：同目录 [crossscale_report.md](#)。